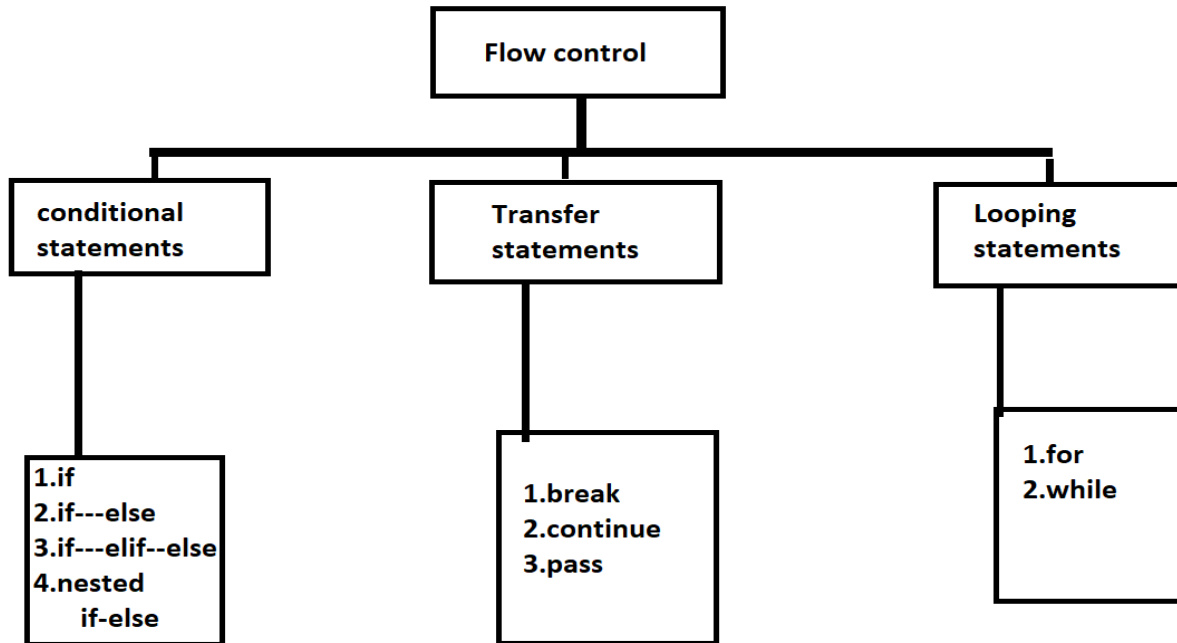


Python

Day-05



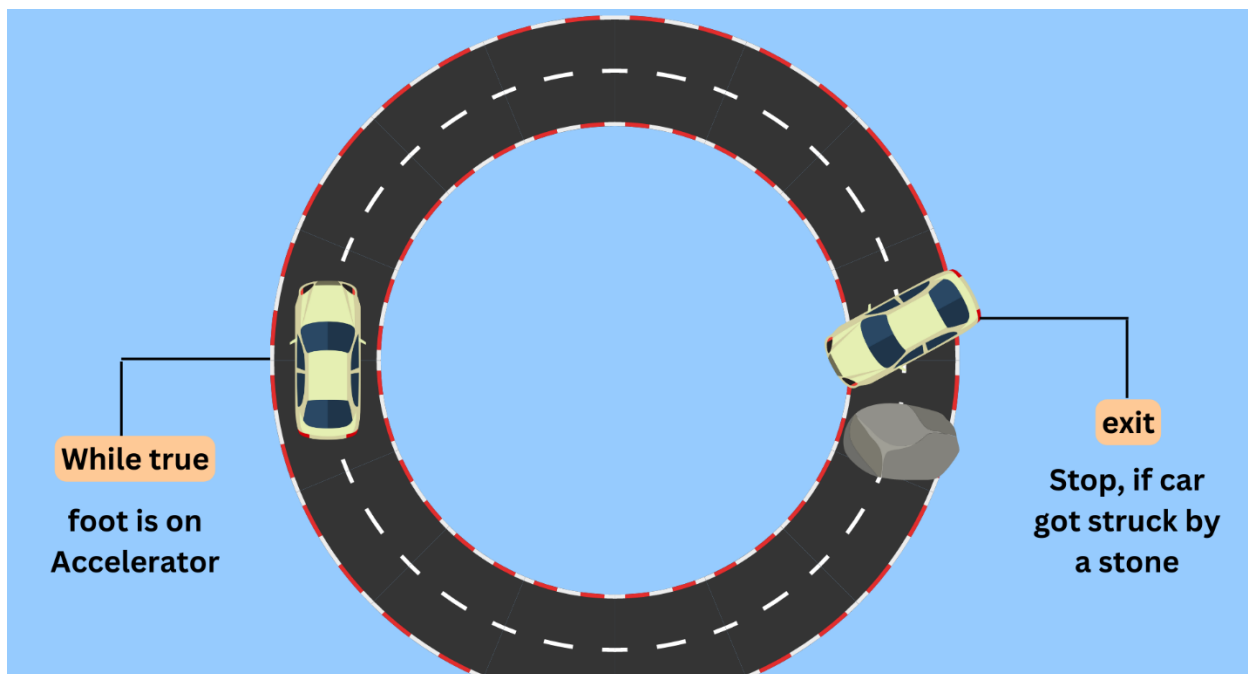
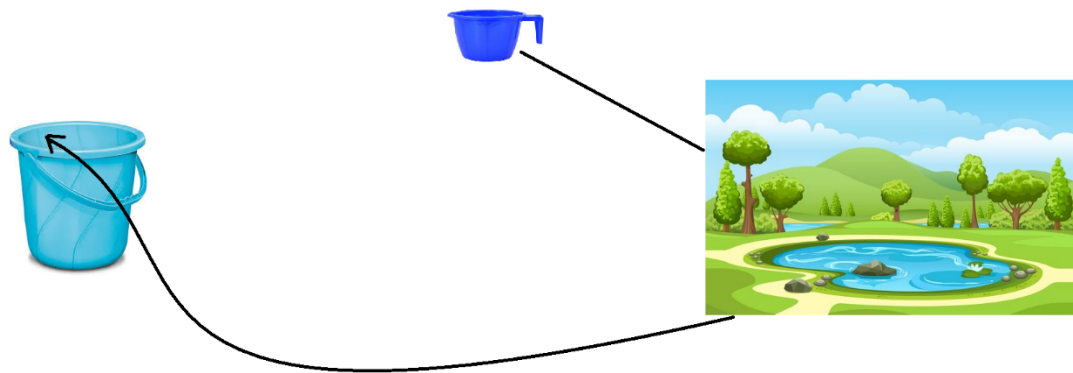
Looping Statements in python

while

for

Definition (**while loop**)

In Python, a while loop is used to execute a block of code repeatedly as long as a given condition is True.



"A while loop keeps doing something again and again until we tell it to stop."

What is **While** Loop in Python?

While loop statement in Python is used to execute statement(s) repeatedly. The number of times the while loop is executed is not known in advance, so the while

loop is called an Indefinite Iteration statement. It will keep on repeatedly executing as long as a while condition is true, and it will stop repeating only if the condition becomes false.

Syntax:

while condition:

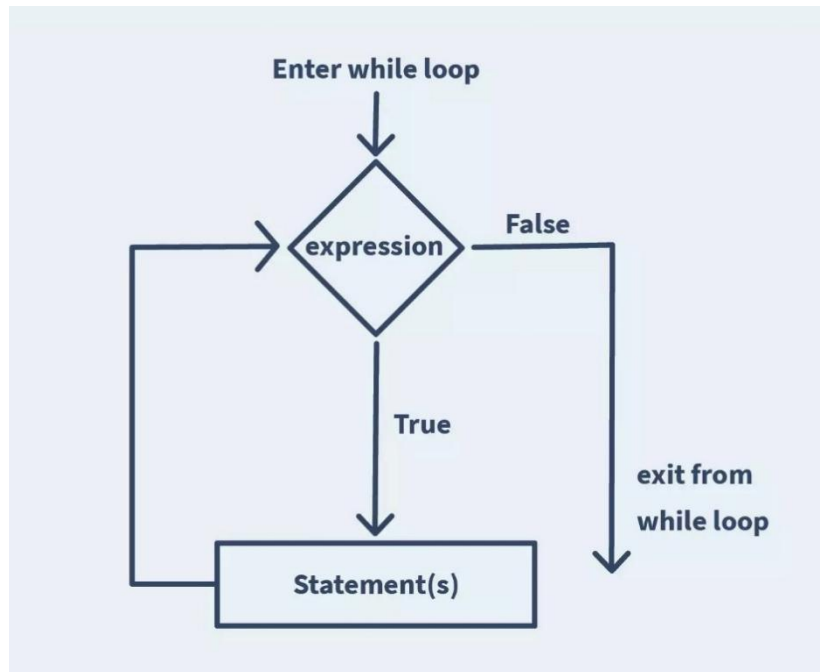
Code to execute

Working of while:

- First, the expression is evaluated. If it returns true, control is passed to the indented statement(s) inside the while loop.
- All the statements that are indented to the right below the while loop for the body of the while loop and are executed.
- Once the statements are executed, control is again passed to the expression, and the expression is re-evaluated. If it returns true, the body is executed again. This repeats until the expression returns false.
- Once the while loop break, control passed to the next unindented line of the while loop

We shall explore the working of while loop more clearly with an example in the next sections of the article.

Flowchart of While Loop in Python



Example of Python While Loop

#1 Write a program to print a name ten times in Python using the while loop

Program:

```
count = 0;
while count < 10:
    print("My name is Vidyut")
    count += 1
```

Output:

```
My name is Vidyut
My name is Vidyut
My name is Vidyut
My name is Vidyut
My name is Vidyut
My name is Vidyut
My name is Vidyut
My name is Vidyut
My name is Vidyut
My name is Vidyut
My name is Vidyut
```

Explanation:

- The objective is to print the “My name is Vidyut” string ten times.
- We took a variable count and initialized it to 0
- The expression ‘count < 10’ is evaluated, and since 0 < 10, control goes inside the while loop
- The ‘print’ statement and count += 1 statement is uniformly indented from the body of the while loop. They are both executed. The string is printed to output once, and the count value increments by 1.
- Control goes back to the expression ‘count < 10’. And since 1 < 10, control again goes inside the while loop.
- This process is repeated till the count value is incremented to 10, and the print statement has been executed a total of 10 times.
- Now, since the condition 10 < 10 returns false, the loop breaks, the body of the while loop is not executed, and the control jumps to the next statement after the while statement.

While Loop with else

While statement in python also supports else statement in association with it. The else statement is optional. Below points illustrate the working of while else:

- The else statement written after the while statement will be executed when the expression of while returns False.
- So, the else statement will be executed once the execution of the while is completed normally(By expression returning False).
- If a break statement is executed inside the while loop and the execution is stopped, then else block is skipped, and control jumps directly to the statement below the while..else.
- Notice the difference between the statements inside the else block and the statements written below the while..else.
 - The statements inside the else block will be skipped if the while loop breaks abruptly. In the normal case of while expression returning false, the else will be executed.
 - The statements below the while..else block will be executed in both the cases when the while expression returns false and also when the while breaks abruptly(using break).

Syntax:

```
while expression:
    statement(s)
else:
    Statement(s)
```

The body of else can be a single statement or a block of uniformly indented statements. The else is executed only once after the while loop condition returns false, and the loop breaks.

Example:

```
count = 0;
while count < 10:
    print("My name is student")
    count += 1
else:
    print("String is printed ten times!")
```

Output:

```
My name is student
My name is student
My name is student
My name is student
My name is student
My name is student
My name is student
My name is student
My name is student
My name is student
My name is student
String is printed 10 times!
```

Explanation:

- The objective is to print the “My name is Vidyut” string ten times.
- We took a variable count and initialized it to 0
- The expression ‘count < 10’ is evaluated, and since $0 < 10$, control goes inside the while loop
- The ‘print’ statement and count += 1 statement, which is uniformly indented from the body of the while loop. They are both executed. The string is printed to output once, and the count value increments by 1.
- Now, control goes back to the expression ‘count < 10’. And since $1 < 10$, control again goes inside the while loop.
- This process is repeated till the count value is incremented to 10, and the print statement has been executed a total of 10 times.
- Now, since the condition $10 < 10$ returns false, the loop breaks, and the body of the while loop is not executed, and control jumps to the next statement after the while statement, which is the else statement.
- Now, the body of else statements gets executed, and the print statement is executed.
- Then, the flow of control goes to the next statement after the while-else statement.

Single Statement While Loop in Python

If there is only one statement in the body of the while loop, the statement can be placed in the same line as the while header statement. This variant is supported in python. The below syntax makes this clear.

```
while expression: statement
```

This is only allowed for a single statement if there are multiple statements, all of them should be written inside the body of the while loop, and that too uniformly indented.

Example:

```
count = 0
while count < 10: print("I run Infinitely")
```

Output:

```
The String "I run Infinitely" getting printed on the console in infinity
```

Explanation:

- The count is initialized to 0
- The expression `count < 10` is evaluated, and the single statement inside the while loop body is executed.
- The control then jumps back to the expression `count < 10`, and it again returns true.
- This process keeps on running infinitely, as the expression `count < 10` will never return false since we are not modifying the count value anywhere.
- So this loop can also be called an Infinite while loop. Let us clearly define and understand what an infinite while loop is in the next section.

The Infinite While Loop in Python

The Infinite while loop is a loop that never stops its repeated execution because the expression (condition) defined in the while header will never return False. The example we saw in the above section is one such infinite while loop.

There can be many use cases of Infinite while loop in programming. A simple instance is a system/program that keeps on running without stopping, like a server program. So that the client program can connect to the server program anytime as long as the server program keeps running.

Let us look at an example Infinite while loop program

```
count = 0
while count != 1:
    name = str(input("What is your name? "))
    print("Hi " + name)
```

Output:

```
What is your name? Rahil
Hi Rahil
What is your name? Raavi
Hi Raavi
What is your name? Rejesh
Hi Rejesh
```

```
What is your name? Traceback (most recent call last):  
  File "main.py", line 3, in <module>  
    name = str(input("What is your name? "))  
KeyboardInterrupt
```

Explanation:

- Here, the expression `count != 1` is always True. So the body of the while loop is executed on infinite.
- This loop keeps asking for input and outputting the value until we hit CTRL + C, which generates a keyboard interrupt to break the loop.

While Loop Control Statements in Python

Loop control statements/ keywords in python written inside a while loop affect the flow of execution of the while loop. They alter the normal execution of a loop. Below are the loop control statements that are supported in python:

Continue Statement:

Continue statement written inside the body of while loop returns the flow of control back to the expression in the header, skipping the execution of the rest of the statements inside the body.

Example:

```
count = 0  
while count < 10:  
    count += 1;  
    if(count == 5): continue  
    print("The value of the count is " + str(count))
```

Output:

```
The value of the count is 1  
The value of the count is 2  
The value of the count is 3  
The value of the count is 4  
The value of the count is 6  
The value of the count is 7  
The value of the count is 8  
The value of the count is 9  
The value of the count is 10
```

Explanation:

- Count is initialized to 0
- `count < 10` is evaluated, and the body of the while is executed.

- When the value of count is equal to 5. The if statement condition `count == 5` will be true. And the statement inside if which is `continue` is executed.
- Once the `continue` statement gets executed, control jumps directly to the while header again, skipping all the statements below it hence the print statement “The value of count...” is not executed in case of `count == 5`
- After the count becomes six again, the flow of control is similar to the normal while loop.

Break Statement:

Break statement inside the body of a while loop will stop the repeated execution and bring the control out of the while loop. It is like an abrupt force stop of a while loop. If it is a `while..else` statement, then the `else` part is also skipped when a `break` statement is executed. Control jumps directly to the statement below the `while..else` statements.

Example:

```
count = 0
while count < 10:
    count += 1;
    if(count == 5): break
    print("The value of the count is " + str(count))
```

Output:

```
The value of the count is 1
The value of the count is 2
The value of the count is 3
The value of the count is 4
```

Explanation:

- Count is initialized to 0
- `count < 10` is evaluated, and the body of the while is executed.
- When the value of count is equal to 5. The if statement condition `count == 5` will be true. And the statement inside if the `break` is executed.
- Once the `break` statement gets executed, control jumps directly out of the while loop, skipping all the statements below it, and the repeated execution of the loop stops. Hence no values equal to and below five got printed as output.

Pass Statement:

A `pass` statement can be seen as a dummy/empty statement. It is used to write empty loops, empty blocks, empty functions, classes, etc. It doesn't really affect the flow of control. It is used to keep the flow seamless.

Example:

```
count = 0
while count < 10:
    count += 1;
    if(count == 5): pass
    print("The value of the count is " + str(count))
```

Output:

```
The value of the count is 1
The value of the count is 2
The value of the count is 3
The value of the count is 4
The value of the count is 5
The value of the count is 6
The value of the count is 7
The value of the count is 8
The value of the count is 9
The value of the count is 10
```

Explanation:

- Count is initialized to 0
- `count < 10` is evaluated, and the body of the while is executed.
- When the value of count is equal to 5. The if statement condition `count == 5` will be true. And the statement inside if which is `pass` is executed.
- In this example, the flow is not disturbed when a `pass` gets executed since it is a dummy statement. The if condition gets evaluated without any errors and control goes to the next statement after the if, which is the print statement.

Learn More:

Questions

Q: What is the while loop in Python?

A: The while loop in Python is a control flow statement that allows you to repeatedly execute a block of code as long as a specified condition is true. It provides a way to create iterative processes in your program.

Q: What is the difference between a while loop and a for loop in Python?

A: The main difference between a while loop and a for loop is the way they control the flow of execution. A while loop repeatedly executes a block of code as long as a condition is true, whereas a for loop iterates over a sequence (such as a list or a range) and executes the block of code for each element in the sequence.

Q: How to avoid infinite loops with while loops?

A: To avoid infinite loops while loops, ensure that the condition within the loop will eventually become False. Otherwise, the loop will continue indefinitely. Make sure to include code within the loop that modifies the variables involved in the condition, allowing the condition to become False at some point.

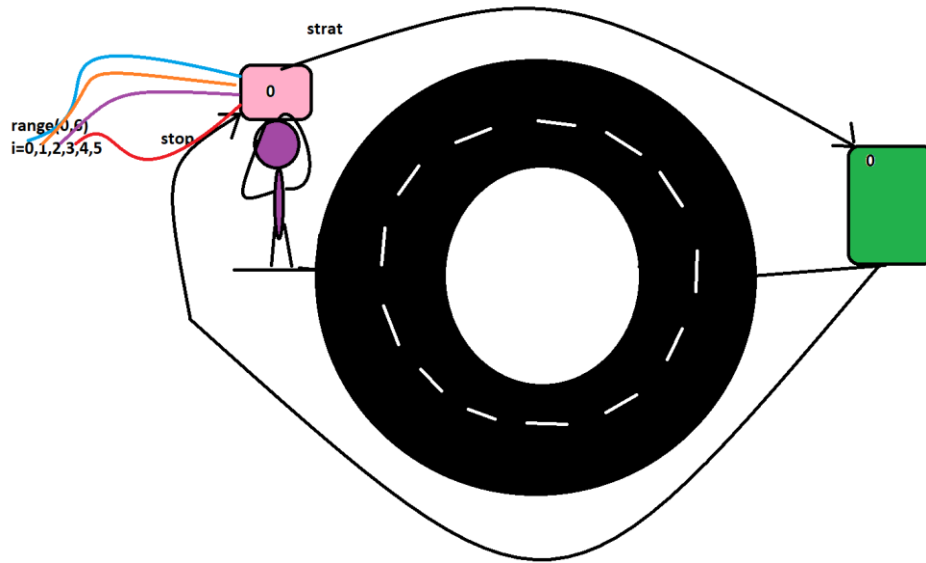
Conclusion

We have reached the end of the article. The examples we took in this article are very basic to give you a clear, in-depth understanding of the while loop in Python. Once you get the hang of the while loop and its variants, you can try to explore the below classic while loop programs in Python.

- Sum all the integers in a given list of python
- Factorial of a number using while loop in Python
- Generate Fibonacci sequence using while loop in Python
- Print the Multiplication table for a given number using the while loop in Python
- Check if the given number is prime or not using the while loop in Python

For loop

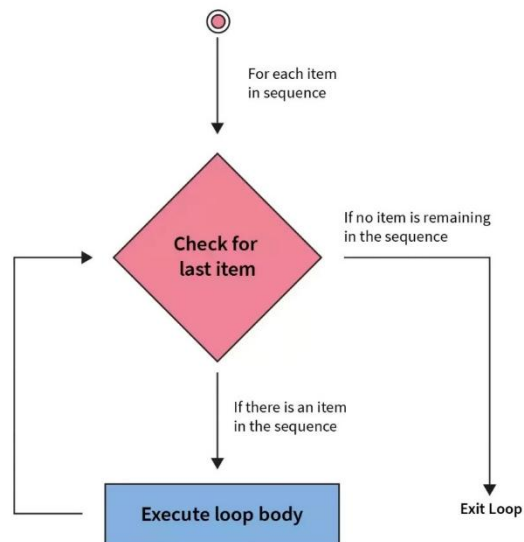
A for loop in Python is used to iterate over a sequence (like a list, tuple, dictionary, set, or string). It is commonly used when you know the number of iterations in advance.



Syntax:

for variable in sequence:

Loop body



Example 1: Iterating Over a List

```
fruits = ["apple", "banana", "cherry"]
```

```
for fruit in fruits:
```

```
    print(fruit)
```

Output:

apple

banana

cherry

Example 2: Using range()

```
for i in range(5): # Loops from 0 to 4
```

```
    print(i)
```

Output:

0

1

2

3

4

Example 3: Iterating Over a String

```
for char in "Python":
```

```
    print(char)
```

Output:

P

y

t

h

o

n

Example 4: Using else with a for loop

```
for i in range(3):
```

```
    print(i)
```

```
else:
```

```
    print("Loop completed")
```

Output:

0

1

2

Loop completed

Nested for Loop



- 1.Monday
- 2.Tuesday
- 3.wendsday
- 4.Thursday
- 5.Friday



Example:

```
for i in range(3):  
    for j in range(2):  
        print(f"i={i}, j={j}")
```

Output:

i=0, j=0

i=0, j=1

i=1, j=0

i=1, j=1

i=2, j=0

i=2, j=1

The range() Function

`range()` is a built-in function provided by Python. This function is commonly used with a for loop for looping over a range of numbers. This function returns a sequence of numbers that, by default, starts with zero, increments by 1, and ends at a specified number passed as an argument. However, it is possible to provide this function as an argument for the starting number, increment step size, and the number at which it needs to stop. There are three ways in which a `range()` function can be used:

range(stop): It takes one argument and is the default version of the `range()` function. It will produce a sequence of numbers starting from 0, incrementing by 1, and ending at the specified number count. For example, `range(15)` will generate the numbers starting at 0 and ending at 14.

range(start, stop): This will take as an argument the starting and the ending numbers and will generate the sequence incrementing by 1. For example, `range(5, 10)` will generate a sequence of numbers starting at 5 and ending at 9, by default incremented by 1.

range(start, stop, step/Increment): This version takes the start value, step value by which the series will increment(positive integer) or decrement(negative integer), and ending values parameters and generates the sequence. For example, `range(6, 21, 2)` will generate all the even numbers between 6 and 21. There will be an increment of 2 in successive values as the step size is 2.

To iterate through a list, `range()` is generally used in combination with the `len()` function. Also, while using the `range()`, we will be accessing the items stored in the sequence by indexing.

Now, let's understand with an example how we can use range() with the for loop in python. In the below code, we are using the for loop to iterate over a list containing the cities using the range() function. Since range() needs to be given at least one argument, i.e., the stop value, the stop value will be the size of the list as we want to print all the cities concatenating with the "I Love" string. So the loop will go through each integer starting from 0 to end (which is the list size) and using indexing, the city name from the list will be printed.

#Program to iterate through a list using range()

```
city = ['London', 'Paris', 'Mumbai', 'Sydney', 'California']
```

```
size = len(city)
```

```
#iterating over the list using the range() function
```

```
for x in range(size):
```

```
    print("I Love", city[x])
```